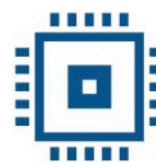




Universitatea *Transilvania* din Brașov
Facultatea de Inginerie Electrică și Știința Calculatoarelor
Departamentul Automatică și Tehnologia Informației



Proiect

P00

BRAȘOV, 2021

Cuprins

1. Proiect.....	2
1.1. Enunț.....	2
1.2. Rezolvare proiect.....	2
1.3. Rezultate	41

1. Proiect

1.1. Enunț

Sa se realizeze o aplicatie C# cu interfata grafica pentru gestiunea unui catalog de note.

Aplicatia va include interfete pentru gestiunea disciplinelor (adaugare, modificare, stergere; datele de interes sunt denumire, cod disciplina, credite), a studentilor (datele de interes sunt: numar matricol, nume, prenume) si a notelor.

Stergerile vor utiliza un flag in baza de date (de exemplu 0=nesters, 1=sters).

Aplicatia va permite generarea unui raport, direct in interfata utilizator continand notele tuturor studentilor la toate disciplinele, impreuna cu media, punctajul si informatia de integralist/ne-integralist. Raportul va putea fi exportat pentru aplicatia MS Excel in format CSV.

discipline

cod_disciplina -> varchar(50) PK
denumire -> varchar(50)
nr_credite -> int
sters -> tinyint default 0 (0=sters, 1=nesters)

studenti

nr_matricol -> int PK
nume -> varchar(50)
prenume -> varchar(50)
sters -> tiny default 0 (0=sters, 1=nesters)

catalog_note

id -> bigint (identitate) - PK A_I
cod_disciplina -> varchar(50) > discipline.cod_disciplina
nr_matricol -> int > studenti.nr_matricol
nota -> int
sters -> tiny default 0 (0=sters, 1=nesters)

STUDENT	DISCIPLINA	NOTA
		SALVARE

VARIANTA 1

STUDENT: xxxx yyyy

DISCIPLINA 1:
DISCIPLINA 2:

.....

SAVE

VARIANTA 2

1.2. Rezolvare temă

În primul rând, pentru a realiza baza de date am utilizat Microsoft SQL Server, care este sistem de gestionare de baze de date relaționale produs de compania americană Microsoft Corp.

Într-o primă etapă am creat tabelele.

DESKTOP-HTA5SRC\...-dbo.Catalog_Nota					
	id	nota	cod_discipl...	nr_matricol	flag_nota
▶	11	6	1	1	0
	12	10	6	1	0
	13	9	7	2	1
	14	10	6	2	0
	15	7	3	2	0
	16	6	1	2	0
	17	5	4	3	0
	18	3	5	3	0
	19	7	3	4	0
	20	8	6	4	0
	21	5	4	5	0
	22	9	6	5	0
	24	10	1	5	0
	25	9	3	4	0
	1002	9	7	1	1
	1003	6	4	4	1
	1004	10	4	1005	0
	1005	10	6	1005	0
*	NULL	NULL	NULL	NULL	NULL

Fig.1.2.1 Tabel Catalog_Note.

DESKTOP-HTA5SRC\S...- dbo.Disciplina				
	cod_discipl...	denumire	nr_credite	flag_discipl...
▶	1	MN	3	1
	3	SAE	4	0
	4	ASCN2	5	1
	5	PS	5	0
	6	POO	3	0
	7	ED	3	0
	1002	TS2	4	0
	1003	TS1	5	0
*	NULL	NULL	NULL	NULL

Fig.1.2.2 Tabel Disciplina.

DESKTOP-HTA5SRC\...log - dbo.Student			
	nr_matricol	nume	flag_student
▶	1		0
	2		0
	3		0
	4		0
	5		0
	1005		1
	1008		1
*	NULL		NULL

Fig.1.2.3 Tabel Student.

Am stabilit legăturile dintre tabele (FK – în tabelul Catalog_Nota) și câteva detalii (autoincrementarea ID-urilor din fiecare tabel).

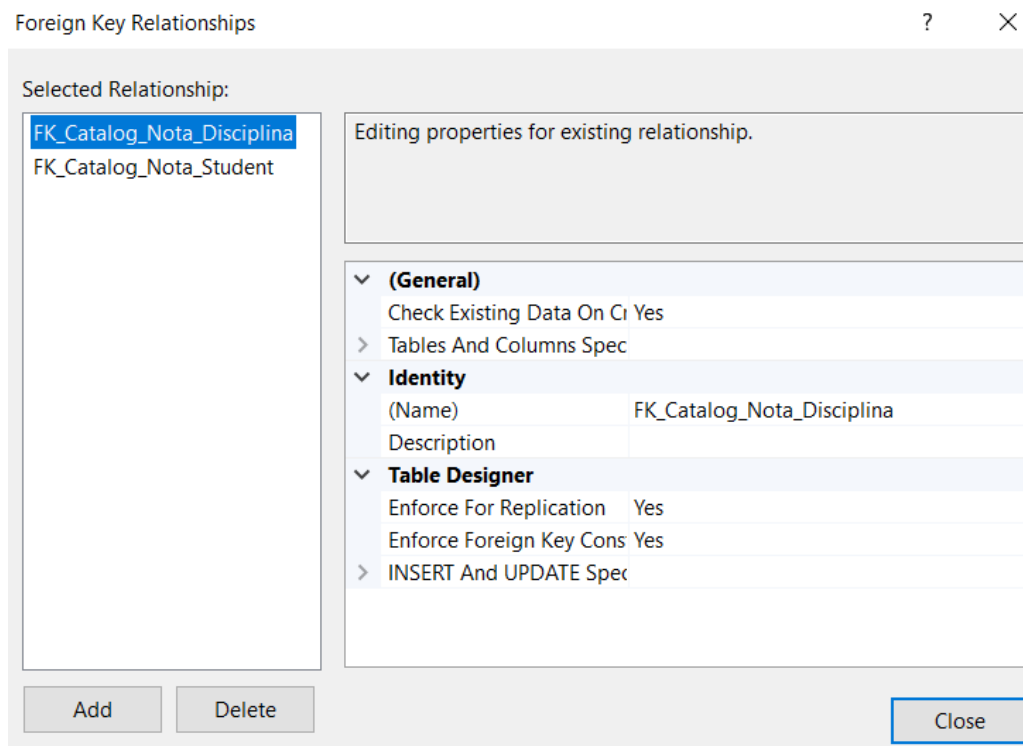


Fig.1.2.4 Realizare legături tabele.

De exemplu, FK cod_disciplina din Catalog_Nota leagă acest tabel de tabelul disciplina prin "PK cod_disciplina".

Am adăugat informații în tabele pt a testa funcționalitatea. În continuare, am creat proceduri pentru a lucra cu informațiile din baza de date.

Câteva dintre procedurile utilizate vor fi prezentate în tabelele următoare:

AdaugaDisciplina

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[AdaugaDisciplina]    Script Date: 6/20/2021
5:19:24 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER PROCEDURE [dbo].[AdaugaDisciplina]
    @denumire varchar(50),
    @nr_credite int,
    @flag int
AS
BEGIN
    insert into Disciplina(denumire, nr_credite, flag_disciplina)
    values (@denumire, @nr_credite, @flag)
END
```

AdaugaNota

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[AdaugaNota]    Script Date: 6/20/2021 5:20:42
PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[AdaugaNota]

@NOTA int,
@COD_DISCIPLINA int,
@NR_MATRICOL int,
@FLAG_NOTA int

AS
BEGIN

insert into Catalog_Nota(cod_disciplina,nr_matricol,nota,flag_nota)
values (@COD_DISCIPLINA, @NR_MATRICOL,@NOTA,@FLAG_NOTA)

END
```

GetAllDiscipline

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[GetAllDiscipline]    Script Date: 6/20/2021
5:22:21 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[GetAllDiscipline]

AS
BEGIN

select * from Disciplina

END

END
```

GetAllNote

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[GetAllNote]    Script Date: 6/20/2021 5:23:53
PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[GetAllNote]

AS
```

```
BEGIN
select * from Catalog_Nota
END
```

GetCrediteSiNotaPtStudent

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[GetCrediteSiNotaPtStudent]    Script Date:
6/20/2021 5:26:00 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER PROCEDURE [dbo].[GetCrediteSiNotaPtStudent]
    @nr_matricol int
AS
BEGIN
    SELECT Catalog_Nota.nota, Disciplina.nr_credite
    FROM Catalog_Nota
    INNER JOIN Disciplina ON
Disciplina.cod_disciplina=Catalog_Nota.cod_disciplina
    INNER JOIN Student ON Student.nr_matricol = Catalog_Nota.nr_matricol
    WHERE Student.nr_matricol = @nr_matricol
END
```

GetDisciplinaSiNotaPtStudent

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[GetDisciplinaSiNotaPtStudent]    Script Date:
6/20/2021 5:26:48 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER PROCEDURE [dbo].[GetDisciplinaSiNotaPtStudent]
    @STUDENT int
AS
BEGIN
    select cod_disciplina, nota from Catalog_Nota
    where nr_matricol=@STUDENT
END
```

ModificaDisciplina

```
USE [Catalog]
GO
/***** Object:  StoredProcedure [dbo].[ModificaDisciplina]    Script Date: 6/20/2021
5:27:34 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[ModificaDisciplina]
```

```
@DENUMIRE varchar(50),
@NR_CREDITE int,
@FLAG_DISCIPLINA int,
@ID int
AS
BEGIN
    update Disciplina
    set denumire=@DENUMIRE,
        nr_credite=@NR_CREDITE,
        flag_disciplina=@FLAG_DISCIPLINA
    where cod_disciplina=@ID
END
```

StergeStudent

```
USE [Catalog]
GO
/***** Object: StoredProcedure [dbo].[StergeStudent]    Script Date: 6/20/2021
5:28:17 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[StergeStudent]

    @ID int,
    @FLAG_STUDENT int
AS
BEGIN
    update Student
    set flag_student=@FLAG_STUDENT
    where nr_matricol=@ID
END
```

Restul procedurilor se află în fișierul "Progamability -> Stored Procedures".

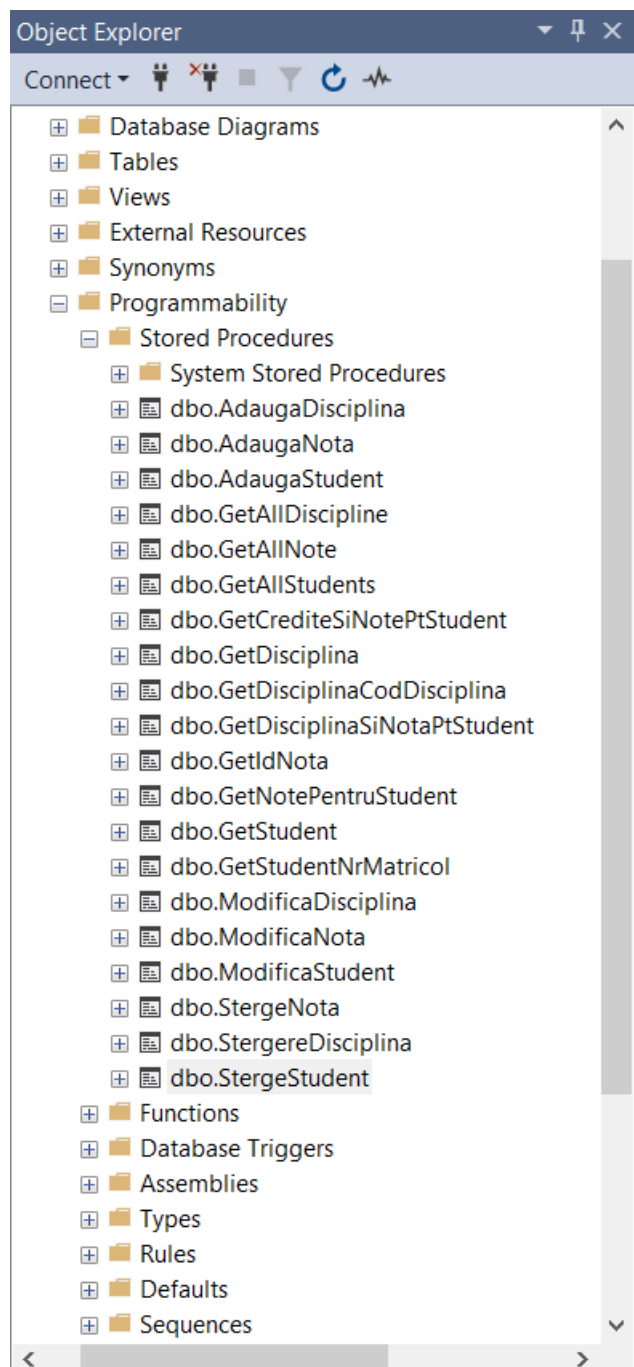


Fig.1.2.5 Procedurile din baza de date.

Pentru a realiza interfetele cu funcționalități am folosit Visual Studio 2019, care include un set complet de instrumente de dezvoltare pentru generarea de aplicații ASP.NET, Servicii Web XML, aplicații desktop și aplicații mobile.

Cu scopul de a realiza conexiunea cu baza de date am folosit un string de conexiune în fișierul "App.config".



Fig.1.2.6 String de conexiune cu baza de date.

În continuare, vor fi prezentate tabele care vor conține coduri din diferite clasei care conduc la realizarea aplicației.

BasePropertyChanged.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Catalog_Note
{
    class BasePropertyChanged
    {
        public event PropertyChangedEventHandler PropertyChanged;
        protected void NotifyPropertyChanged(string propertyName)
        {
            if (PropertyChanged != null)
                PropertyChanged(this, new PropertyChangedEventArgs(propertyName));
        }
    }
}
```

DbConnector.cs

```
using System;
using System.Collections.Generic;
using System.Configuration;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Catalog_Note
{
    //conectam baza de date de codul C#
    class DbConnector
    {
        private static readonly string connectionString =
            ConfigurationManager.ConnectionStrings["myConStr"].ConnectionString;

        public static SqlConnection Connection
        {
            get
            {
                return new SqlConnection(connectionString);
            }
        }
    }
}
```

```
        {  
            return new SqlConnection(connectionString);  
        }  
    }  
}
```

Disciplina.cs

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace Catalog_Note  
{  
    //clasa Disciplina reprezinta tabelul Disciplina din baza de date  
    //fiecare coloana din acel tabel este reprezentat prin metode cu setter si getteri  
    //NotifyPropertyChanged - anunta cand se modifica ceva  
    class Disciplina :BasePropertyChanged  
    {  
        //?- in baza de date se autoincrementeaza  
        private int? codDisciplina;  
        public int? CodDisciplina  
        {  
            get  
            {  
                return codDisciplina;  
            }  
            set  
            {  
                codDisciplina = value;  
                NotifyPropertyChanged("CodDisciplina");  
            }  
        }  
  
        private string denumire;  
        public string Denumire  
        {  
            get  
            {  
                return denumire;  
            }  
            set  
            {  
                denumire = value;  
                NotifyPropertyChanged("Denumire");  
            }  
        }  
  
        private int nrCredite;  
        public int NrCredite  
        {  
            get  
            {  
                return nrCredite;  
            }  
            set  
            {  
                nrCredite = value;  
                NotifyPropertyChanged("NrCredite");  
            }  
        }  
    }  
}
```

```
        nrCredite = value;
        NotifyPropertyChanged("NrCredite");
    }
}

private int flagDisciplina;
public int FlagDisciplina
{
    get
    {
        return flagDisciplina;
    }
    set
    {
        flagDisciplina = value;
        NotifyPropertyChanged("FlagDisciplina");
    }
}
}
```

Nota.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Catalog_Note
{
    //clasa Disciplinana reprezinta tabelul Disciplina din baza de date
    //fiecare coloana din acel tabel este reprezentat prin metode cu setter si getteri
    //NotifyPropertyChanged - anunta cand se modifica ceva
    class Disciplina :BasePropertyChanged
    {
        //?- in baza de date se autoincrementeaza
        private int? codDisciplina;
        public int? CodDisciplina
        {
            get
            {
                return codDisciplina;
            }
            set
            {
                codDisciplina = value;
                NotifyPropertyChanged("CodDisciplina");
            }
        }

        private string denumire;
        public string Denumire
        {
            get
            {
                return denumire;
            }
        }
    }
}
```

```
        }
        set
        {
            denumire = value;
            NotifyPropertyChanged("Denumire");
        }
    }

    private int nrCredite;
    public int NrCredite
    {
        get
        {
            return nrCredite;
        }
        set
        {
            nrCredite = value;
            NotifyPropertyChanged("NrCredite");
        }
    }

    private int flagDisciplina;
    public int FlagDisciplina
    {
        get
        {
            return flagDisciplina;
        }
        set
        {
            flagDisciplina = value;
            NotifyPropertyChanged("FlagDisciplina");
        }
    }
}
}
```

Student.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Catalog_Note
{
    class Student:BasePropertyChanged
    {
        private int? nrMatricol;
        public int? NrMatricol
        {
            get
            {
                return nrMatricol;
            }
        }
    }
}
```

```
        }
        set
        {
            nrMatricol = value;
            NotifyPropertyChanged("nrMatricol");
        }
    }

    private string nume;
    public string Nume
    {
        get
        {
            return nume ;
        }
        set
        {
            nume = value;
            NotifyPropertyChanged("Nume");
        }
    }

    private int flagStudent;
    public int FlagStudent
    {
        get
        {
            return flagStudent;
        }
        set
        {
            flagStudent = value;
            NotifyPropertyChanged("FlagStudent");
        }
    }
}
}
```

ListaDiscipline.cs

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Data;
using System.Data.SqlClient;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Catalog_Note
{
    class ListaDiscipline
    {
        //ObservableCollection -tip de lista care notifica atunci cand se modifica
        public ObservableCollection<Disciplina> Discipline
    }
```

```
{
    get;

    set;
}

public ListaDiscipline()
{
    Discipline = new ObservableCollection<Disciplina>()
    {
        new Disciplina()
    };
}

//functia corespunzatoare procedurii din baza de date cu acelasi nume menita
sa returneze toate disciplinele existente
public void GetAllDiscipline()
{
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("GetAllDiscipline", con);
        cmd.CommandType = CommandType.StoredProcedure;
        con.Open();
        SqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            Discipline.Add(new Disciplina()
            {
                CodDisciplina = (int)(reader[0]),
                Denumire = reader.GetString(1),
                NrCredite = (int)(reader[2]),
                FlagDisciplina = (int)(reader[3])
            });
        }
        reader.Close();
    }
    finally
    {
        con.Close();
    }
}
}
```

Tabelul "Note" contine doua FK-uri(cod_disciplina,nr_matricol). Pentru a putea afisa in fereastra dedicata notelor(ViewNote) numele disciplinei si numele studentului care au FK-urile specificate a trebuit sa creez doua functii denumite GetDisciplina, GetStudent. Acestea au, de asemenea, proceduri in baza de date, iar totul s-a rezumat la a folosi ceea ce aceste proceduri returneaza.

ListaNote.cs
<pre>using System; using System.Collections.Generic; using System.Collections.ObjectModel;</pre>

```
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;

namespace Catalog_Note
{
    class ListaNote
    {
        public ObservableCollection<Nota> Note
        {
            get;
            set;
        }

        public ListaNote()
        {
            Note = new ObservableCollection<Nota>()
            {
                new Nota()
            };
        }

        public void GetAllNote()
        {
            SqlConnection con = DbConnecter.Connection;
            try
            {
                SqlCommand cmd = new SqlCommand("GetAllNote", con);
                cmd.CommandType = CommandType.StoredProcedure;
                con.Open();
                SqlDataReader reader = cmd.ExecuteReader();
                while (reader.Read())
                {
                    int codDisciplina = (int)(reader[2]);
                    int nrMatricol = (int)(reader[3]);

                    Note.Add(new Nota()
                    {
                        Id = (int)(reader[0]),
                        NotaStudent = (int)(reader[1]),
                        Disciplina = GetDisciplina(codDisciplina),
                        Student = GetStudent(nrMatricol),
                        FlagNota = (int)(reader[4])
                    });
                }
                reader.Close();
            }
            finally
            {
                con.Close();
            }
        }

        private string GetStudent(int nrMatricol)
        {

```

```
SqlConnection con = DbConnector.Connection;
SqlDataReader reader = null;
try
{
    SqlCommand cmd = new SqlCommand("GetStudent", con);
    cmd.CommandType = CommandType.StoredProcedure;

    var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);

    cmd.Parameters.Add(paramNrMatricol);

    con.Open();

    reader = cmd.ExecuteReader();
    reader.Read();

    string student;
    student = reader.GetString(0);
    return student;
}
catch
{
    MessageBox.Show("Error In DataBase");
    return "";
}
finally
{
    reader.Close();
}
}

public string GetDisciplina(int codDisciplina)
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetDisciplina", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);

        cmd.Parameters.Add(paramCodDisciplina);

        con.Open();

        reader = cmd.ExecuteReader();
        reader.Read();

        string disciplina;
        disciplina = reader.GetString(0);
        return disciplina;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return "";
    }
    finally

```

```
        {  
            reader.Close();  
        }  
    }  
}
```

ListaStudent.cs

```
using System;  
using System.Collections.Generic;  
using System.Collections.ObjectModel;  
using System.Data;  
using System.Data.SqlClient;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace Catalog_Note  
{  
    class ListaStudent  
    {  
        public ObservableCollection<Student> Studenti  
        {  
            get;  
  
            set;  
        }  
  
        public ListaStudent()  
        {  
            Studenti = new ObservableCollection<Student>()  
            {  
                new Student()  
            };  
        }  
  
        public void GetAllStudenti()  
        {  
            SqlConnection con = DbConnector.Connection;  
            try  
            {  
                SqlCommand cmd = new SqlCommand("GetAllStudents", con);  
                cmd.CommandType = CommandType.StoredProcedure;  
                con.Open();  
                SqlDataReader reader = cmd.ExecuteReader();  
                while (reader.Read())  
                {  
                    Studenti.Add(new Student()  
                    {  
                        NrMatricol = (int)(reader[0]),  
                        Nume = reader.GetString(1),  
                        FlagStudent = (int)(reader[2])  
                    });  
                }  
                reader.Close();  
            }  
            finally  
            {  

```

```
        con.Close();  
    }  
    }  
}
```

Raportul se referea a afișa informații pentru un anumit student selectat în fereastra ViewRaport. De aceea, lista raportului ("Raport") este de tipul "Student".

IblStudenti_SelectionChanged – se apelează ori de câte ori se selectează un obiect din lista de student din ViewRaport. La selectare, am dorit să apară detaliile acestui student (disciplină, notă, nr_credite, media, integralist/neintegralist). Pentru a accesa aceste informații au fost necesare mai multe funcții, fiecare având proceduri în baza de date:

- GetDisciplinaSiNotaPtStudent – returnează o listă de string-uri în formatul "Disciplină, Notă". Procedura din baza de date returnează o listă pe fiecare poziție având 2 valori: Cod disciplină, Notă. În C#, aveam nevoie de numele disciplinei, de aceea am apelat procedura care returnează numele disciplinei specifice unui cod. Pentru notă am transformat-o în string și am format din cele două informații stringul precizat anterior.
- GetNrCrediteSiMedie – returnează un Tuple<int,double> în care am reținut numărul de credite și media
- GetIntegralist

ListaRaport.cs

```
using System;  
using System.Collections.Generic;  
using System.Collections.ObjectModel;  
using System.Data;  
using System.Data.SqlClient;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows;  
  
namespace Catalog_Note  
{  
    class ListaRaport  
    {  
        public ObservableCollection<Student> Raport  
        {  
            get;  
            set;  
        }  
  
        public ListaRaport()  
        {  
            Raport = new ObservableCollection<Student>()  
        }  
    }  
}
```

```
        {
            new Student()
        };
    }

    public void GetAllStudenti()
    {
        SqlConnection con = DbConnecter.Connection;
        try
        {
            SqlCommand cmd = new SqlCommand("GetAllStudents", con);
            cmd.CommandType = CommandType.StoredProcedure;
            con.Open();
            SqlDataReader reader = cmd.ExecuteReader();
            while (reader.Read())
            {
                Raport.Add(new Student()
                {
                    NrMatricol = (int)(reader[0]),
                    Nume = reader.GetString(1),
                    FlagStudent = (int)(reader[2])
                });
            }
            reader.Close();
        }
        finally
        {
            con.Close();
        }
    }
}
```

În continuare, vor fi prezentate ferestrele asociate fiecărei funcționalități cerute în cerință.

ViewDiscipline.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace Catalog_Note
{
    /// <summary>
    /// Interaction logic for ViewDiscipline.xaml

```

```
/// </summary>
public partial class ViewDiscipline : Window
{
    public ViewDiscipline()
    {
        InitializeComponent();
        (DataContext as ListaDiscipline).GetAllDiscipline();
    }

    private void lblCodDisciplina_SelectedIndexChanged(object sender,
        SelectionChangedEventArgs e)
    {
        if (lblCodDisciplina.SelectedIndex != -1)
        {
            txtCodDisciplina.Text = (lblCodDisciplina.SelectedItem as
                Disciplina).CodDisciplina.ToString();
            txtDenumire.Text = (lblCodDisciplina.SelectedItem as
                Disciplina).Denumire;
            txtNrCredite.Text = (lblCodDisciplina.SelectedItem as
                Disciplina).NrCredite.ToString();
            txtFlag.Text = (lblCodDisciplina.SelectedItem as
                Disciplina).FlagDisciplina.ToString();
        }
    }

    private void btnAdauga_Click(object sender, RoutedEventArgs e)
    {
        SqlConnection con = DbConnector.Connection;
        //SqlDataReader reader = null;
        try
        {
            SqlCommand cmd = new SqlCommand("AdaugaDisciplina", con);
            cmd.CommandType = CommandType.StoredProcedure;

            var paramDenumire = new SqlParameter("@denumire", txtDenumire.Text);
            var paramNrCredite = new SqlParameter("@nr_credite",
                int.Parse(txtNrCredite.Text));
            var paramFlag = new SqlParameter("@flag", int.Parse(txtFlag.Text));

            cmd.Parameters.Add(paramDenumire);
            cmd.Parameters.Add(paramNrCredite);
            cmd.Parameters.Add(paramFlag);

            con.Open();

            cmd.ExecuteNonQuery();
            MessageBox.Show("Disciplina has been added! Check DataBase!");
        }
        catch
        {
            MessageBox.Show("Error In DataBase");
        }

        int codDisciplina;
        codDisciplina = GetDisciplinaCodDisciplina();
        (DataContext as ListaDiscipline).Discipline.Add(new Disciplina()
        {
            CodDisciplina = codDisciplina,
            Denumire = txtDenumire.Text,
            NrCredite=int.Parse(txtNrCredite.Text),

```

```
        FlagDisciplina = int.Parse(txtFlag.Text)

    });
}

public int GetDisciplinaCodDisciplina()
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetDisciplinaCodDisciplina", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramDenumire = new SqlParameter("@DENUMIRE", txtDenumire.Text);

        cmd.Parameters.Add(paramDenumire);

        con.Open();

        reader = cmd.ExecuteReader();
        reader.Read();

        int codDisciplina;
        codDisciplina = (int)reader[0];
        return codDisciplina;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return -1;
    }
    finally
    {
        reader.Close();
    }
}

private void btnModifica_Click(object sender, RoutedEventArgs e)
{
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("ModificaDisciplina", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramDenumire = new SqlParameter("@DENUMIRE", txtDenumire.Text);
        var paramNrCredite = new SqlParameter("@NR_CREDITE",
int.Parse(txtNrCredite.Text));
        var paramFlagDisciplina = new SqlParameter("@FLAG_DISCIPLINA",
int.Parse(txtFlag.Text));
        var paramID= new SqlParameter("@ID", int.Parse(txtCodDisciplina.Text));

        cmd.Parameters.Add(paramDenumire);
        cmd.Parameters.Add(paramNrCredite);
        cmd.Parameters.Add(paramFlagDisciplina);
        cmd.Parameters.Add(paramID);

        con.Open();
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
    }
    finally
    {
        reader.Close();
    }
}
```

```
        cmd.ExecuteNonQuery();

        MessageBox.Show("Disciplina has been updated! Check DataBase!");
    }
    catch
    {
        MessageBox.Show("Error in DataBase!");
    }

    //cauta in lista obiectul x dupa conditia specificata dupa "="
    var item = (DataContext as ListaDiscipline).Discipline.FirstOrDefault(x =>
x.CodDisciplina == int.Parse(txtCodDisciplina.Text));
    //returneaza indexul din lista al obiectului gasit anterior
    int index = (DataContext as ListaDiscipline).Discipline.IndexOf(item);
    (DataContext as ListaDiscipline).Discipline[index] = new Disciplina()
    {
        CodDisciplina = int.Parse(txtCodDisciplina.Text),
        Denumire = txtDenumire.Text,
        FlagDisciplina= int.Parse(txtFlag.Text),
        NrCredite=int.Parse(txtNrCredite.Text)
    };
}

private void btnSterge_Click(object sender, RoutedEventArgs e)
{
    int flag = 1;
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("StergereDisciplina", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramFlagDisciplina = new SqlParameter("@FLAG_DISCIPLINA", flag);
        var paramID = new SqlParameter("@ID",
int.Parse(txtCodDisciplina.Text));

        cmd.Parameters.Add(paramFlagDisciplina);
        cmd.Parameters.Add(paramID);

        con.Open();

        cmd.ExecuteNonQuery();

        MessageBox.Show("Disciplina has been deleted(modified flag)! Check
DataBase!");
    }
    catch
    {
        MessageBox.Show("Error in DataBase!");
    }

    var item = (DataContext as ListaDiscipline).Discipline.FirstOrDefault(x =>
x.CodDisciplina== int.Parse(txtCodDisciplina.Text));
    int index = (DataContext as ListaDiscipline).Discipline.IndexOf(item);
    (DataContext as ListaDiscipline).Discipline[index] = new Disciplina()
    {
        CodDisciplina = int.Parse(txtCodDisciplina.Text),
        Denumire = txtDenumire.Text,
        FlagDisciplina = flag,
        NrCredite = int.Parse(txtNrCredite.Text)
    }
}
```

```
        }  
    };  
  
    private void btnBack_Click(object sender, RoutedEventArgs e)  
    {  
        MainWindow mainWindow = new MainWindow();  
        mainWindow.Show();  
        this.Close();  
    }  
}
```

ViewNote.cs

```
using System;  
using System.Collections.Generic;  
using System.Data;  
using System.Data.SqlClient;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using System.Windows;  
using System.Windows.Controls;  
using System.Windows.Data;  
using System.Windows.Documents;  
using System.Windows.Input;  
using System.Windows.Media;  
using System.Windows.Media.Imaging;  
using System.Windows.Shapes;  
  
namespace Catalog_Note  
{  
    /// <summary>  
    /// Interaction logic for ViewNote.xaml  
    /// </summary>  
    public partial class ViewNote : Window  
    {  
        public ViewNote()  
        {  
            InitializeComponent();  
            (DataContext as ListaNote).GetAllNote();  
        }  
  
        private void lblID_SelectedIndexChanged(object sender,  
SelectionChangedEventArgs e)  
        {  
            if (lblID.SelectedIndex != -1)  
            {  
                txtID.Text = (lblID.SelectedItem as Nota).Id.ToString();  
                txtNota.Text = (lblID.SelectedItem as Nota).NotaStudent.ToString();  
                txtDisciplina.Text = (lblID.SelectedItem as Nota).Disciplina;  
                txtStudent.Text = (lblID.SelectedItem as Nota).Student;  
                txtFlag.Text = (lblID.SelectedItem as Nota).FlagNota.ToString();  
            }  
        }  
  
        public int GetDisciplinaCodDisciplina()  
        {  
            SqlConnection con = DbConnecter.Connection;  
            SqlDataReader reader = null;
```

```
        try
        {
            SqlCommand cmd = new SqlCommand("GetDisciplinaCodDisciplina", con);
            cmd.CommandType = CommandType.StoredProcedure;

            var paramDenumire = new SqlParameter("@DENUMIRE", txtDisciplina.Text);

            cmd.Parameters.Add(paramDenumire);

            con.Open();

            reader = cmd.ExecuteReader();
            reader.Read();

            int codDisciplina;
            codDisciplina = (int)(reader[0]);
            return codDisciplina;
        }
        catch
        {
            MessageBox.Show("Error In DataBase");
            return -1;
        }
        finally
        {
            reader.Close();
        }
    }

    private int GetStudenNrMatricol()
    {
        SqlConnection con = DbConnector.Connection;
        SqlDataReader reader = null;
        try
        {
            SqlCommand cmd = new SqlCommand("GetStudentNrMatricol", con);
            cmd.CommandType = CommandType.StoredProcedure;

            var paramNume = new SqlParameter("@NAME", txtStudent.Text);

            cmd.Parameters.Add(paramNume);

            con.Open();

            reader = cmd.ExecuteReader();
            reader.Read();

            int id;
            id = (int)(reader[0]);
            return id;
        }
        catch
        {
            MessageBox.Show("Error In DataBase");
            return -1;
        }
        finally
        {
            reader.Close();
        }
    }
}
```

```
    }

    private int GetIdNota(int codDisciplina, int nrMatricol)
    {
        SqlConnection con = DbConnector.Connection;
        SqlDataReader reader = null;
        try
        {
            SqlCommand cmd = new SqlCommand("GetIdNota", con);
            cmd.CommandType = CommandType.StoredProcedure;

            var paramNota = new SqlParameter("@NOTA", int.Parse(txtNota.Text));
            var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);
            var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);
            var paramFlag = new SqlParameter("@FLAG", int.Parse(txtFlag.Text));

            cmd.Parameters.Add(paramNota);
            cmd.Parameters.Add(paramCodDisciplina);
            cmd.Parameters.Add(paramNrMatricol);
            cmd.Parameters.Add(paramFlag);

            con.Open();

            reader = cmd.ExecuteReader();
            reader.Read();

            int id;
            id = (int)(reader[0]);
            return id;
        }
        catch
        {
            MessageBox.Show("Error In DataBase");
            return -1;
        }
        finally
        {
            reader.Close();
        }
    }

    private void btnAdauga_Click(object sender, RoutedEventArgs e)
    {
        int codDisciplina = GetDisciplinaCodDisciplina();
        int nrMatricol = GetStudenNrMatricol();
        SqlConnection con = DbConnector.Connection;
        try
        {
            SqlCommand cmd = new SqlCommand("AdaugaNota", con);
            cmd.CommandType = CommandType.StoredProcedure;

            var paramNota = new SqlParameter("@NOTA", txtNota.Text);
            var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);
            var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);
            var paramFlag = new SqlParameter("@FLAG_NOTA",
int.Parse(txtFlag.Text));

            cmd.Parameters.Add(paramNota);
            cmd.Parameters.Add(paramCodDisciplina);
```

```
        cmd.Parameters.Add(paramNrMatricol);
        cmd.Parameters.Add(paramFlag);

        con.Open();

        cmd.ExecuteNonQuery();
        MessageBox.Show("Nota has been added! Check DataBase!");
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
    }
    int id = GetIdNota(codDisciplina, nrMatricol);
    (DataContext as ListaNote).Note.Add(new Nota()
    {
        Id = id,
        NotaStudent = int.Parse(txtNota.Text),
        Disciplina = txtDisciplina.Text,
        Student = txtStudent.Text,
        FlagNota = int.Parse(txtFlag.Text)
    }); ;
}

private void btnModifica_Click(object sender, RoutedEventArgs e)
{
    int codDisciplina = GetDisciplinaCodDisciplina();
    int nrMatricol = GetStudenNrMatricol();
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("ModificaNota", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNota = new SqlParameter("@NOTA", int.Parse(txtNota.Text));
        var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);
        var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);
        var paramFlagNota = new SqlParameter("@FLAG_NOTA",
int.Parse(txtFlag.Text));
        var paramID = new SqlParameter("@ID", int.Parse(txtID.Text));

        cmd.Parameters.Add(paramNota);
        cmd.Parameters.Add(paramCodDisciplina);
        cmd.Parameters.Add(paramNrMatricol);
        cmd.Parameters.Add(paramFlagNota);
        cmd.Parameters.Add(paramID);

        con.Open();

        cmd.ExecuteNonQuery();

        MessageBox.Show("Nota has been updated! Check DataBase!");
    }
    catch
    {
        MessageBox.Show("Error in DataBase!");
    }

    var item = (DataContext as ListaNote).Note.FirstOrDefault(x => x.Id ==
int.Parse(txtID.Text));
```

```
        int index = (DataContext as ListaNote).Note.IndexOf(item);
        (DataContext as ListaNote).Note[index] = new Nota()
        {
            Id = int.Parse(txtID.Text),
            NotaStudent = int.Parse(txtNota.Text),
            Disciplina = txtDisciplina.Text,
            Student = txtStudent.Text,
            FlagNota = int.Parse(txtFlag.Text)
        };
    }

    private void btnSterge_Click(object sender, RoutedEventArgs e)
    {
        int flag = 1;
        SqlConnection con = DbConnecter.Connection;
        try
        {
            SqlCommand cmd = new SqlCommand("StergeNota", con);
            cmd.CommandType = CommandType.StoredProcedure;

            var paramFlagNota = new SqlParameter("@FLAG_NOTA", flag);
            var paramID = new SqlParameter("@ID", int.Parse(txtID.Text));

            cmd.Parameters.Add(paramFlagNota);
            cmd.Parameters.Add(paramID);

            con.Open();

            cmd.ExecuteNonQuery();

            MessageBox.Show("Nota has been deleted(modified flag)! Check
DataBase!");
        }
        catch
        {
            MessageBox.Show("Error in DataBase!");
        }

        var item = (DataContext as ListaNote).Note.FirstOrDefault(x => x.Id ==
int.Parse(txtID.Text));
        int index = (DataContext as ListaNote).Note.IndexOf(item);
        (DataContext as ListaNote).Note[index] = new Nota()
        {
            Id = int.Parse(txtID.Text),
            NotaStudent = int.Parse(txtNota.Text),
            Disciplina = txtDisciplina.Text,
            Student = txtStudent.Text,
            FlagNota = flag
        };
    }

    private void btnBack_Click(object sender, RoutedEventArgs e)
    {
        MainWindow mainWindow = new MainWindow();
        mainWindow.Show();
        this.Close();
    }
}
```

ViewStudent.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace Catalog_Note
{
    /// <summary>
    /// Interaction logic for ViewStudent.xaml
    /// </summary>
    public partial class ViewStudent : Window
    {
        public ViewStudent()
        {
            InitializeComponent();
            (DataContext as ListaStudent).GetAllStudenti();
        }

        private void lblNrMatricol_SelectedIndexChanged(object sender, SelectionChangedEventArgs e)
        {
            if(lblNrMatricol.SelectedIndex!=-1)
            {
                txtNrMatricol.Text = (lblNrMatricol.SelectedItem as Student).NrMatricol.ToString();
                txtName.Text = (lblNrMatricol.SelectedItem as Student).Nume;
                txtFlag.Text= (lblNrMatricol.SelectedItem as Student).FlagStudent.ToString();
            }
        }

        private void btnAdauga_Click(object sender, RoutedEventArgs e)
        {
            SqlConnection con = DbConnector.Connection;
            //SqlDataReader reader = null;
            try
            {
                SqlCommand cmd = new SqlCommand("AdaugaStudent", con);
                cmd.CommandType = CommandType.StoredProcedure;

                var paramNume = new SqlParameter("@NUME", txtName.Text);
                var paramFlag = new SqlParameter("@FLAG_STUDENT", txtFlag.Text);

                cmd.Parameters.Add(paramNume);
                cmd.Parameters.Add(paramFlag);

                con.Open();
            }
            catch { }
        }
    }
}
```

```
        cmd.ExecuteNonQuery();
        MessageBox.Show("Student has been added! Check DataBase!");
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
    }

    int nrMatricol;
    nrMatricol = GetStudenNrMatricol();
    (DataContext as ListaStudent).Studenti.Add(new Student()
    {
        NrMatricol = nrMatricol,
        Nume=txtName.Text,
        FlagStudent=int.Parse(txtFlag.Text)
    });
}

private int GetStudenNrMatricol()
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetStudentNrMatricol", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNume = new SqlParameter("@NAME", txtName.Text);

        cmd.Parameters.Add(paramNume);

        con.Open();

        reader = cmd.ExecuteReader();
        reader.Read();

        int id;
        id = (int)(reader[0]);
        return id;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return -1;
    }
    finally
    {
        reader.Close();
    }
}

private void btnModifica_Click(object sender, RoutedEventArgs e)
{
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("ModificaStudent", con);
```

```
cmd.CommandType = CommandType.StoredProcedure;

var paramNume = new SqlParameter("@NUME", txtName.Text);
var paramFlagStudent = new SqlParameter("@FLAG_STUDENT",
int.Parse(txtFlag.Text));
var paramNrMatricol = new SqlParameter("@NUMAR_MATRICOL",
int.Parse(txtNrMatricol.Text));

cmd.Parameters.Add(paramNume);
cmd.Parameters.Add(paramFlagStudent);
cmd.Parameters.Add(paramNrMatricol);

con.Open();

cmd.ExecuteNonQuery();

MessageBox.Show("Student has been updated! Check DataBase!");
}
catch
{
    MessageBox.Show("Error in DataBase!");
}

var item = (DataContext as ListaStudent).Studenti.FirstOrDefault(x =>
x.NrMatricol == int.Parse(txtNrMatricol.Text));
int index = (DataContext as ListaStudent).Studenti.IndexOf(item);
(DataContext as ListaStudent).Studenti[index] = new Student()
{
    NrMatricol = int.Parse(txtNrMatricol.Text),
    Nume = txtName.Text,
    FlagStudent = int.Parse(txtFlag.Text)
};
}

private void btnSterge_Click(object sender, RoutedEventArgs e)
{
    int flag = 1;
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("StergeStudent", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNrMatricol = new SqlParameter("@ID",
int.Parse(txtNrMatricol.Text));
        var paramFlagStudent = new SqlParameter("@FLAG_STUDENT", flag);

        cmd.Parameters.Add(paramNrMatricol);
        cmd.Parameters.Add(paramFlagStudent);

        con.Open();

        cmd.ExecuteNonQuery();

        MessageBox.Show("Student has been deleted(modified flag)! Check
DataBase!");
    }
    catch
```

```
        {
            MessageBox.Show("Error in DataBase!");
        }

        var item = (DataContext as ListaStudent).Studenti.FirstOrDefault(x =>
x.NrMatricol == int.Parse(txtNrMatricol.Text));
        int index = (DataContext as ListaStudent).Studenti.IndexOf(item);
        (DataContext as ListaStudent).Studenti[index] = new Student()
        {
            NrMatricol = int.Parse(txtNrMatricol.Text),
            Nume = txtName.Text,
            FlagStudent = flag
        };
    }

    private void btnBack_Click(object sender, RoutedEventArgs e)
    {
        MainWindow mainWindow = new MainWindow();
        mainWindow.Show();
        this.Close();
    }
}
```

ViewNote.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace Catalog_Note
{
    /// <summary>
    /// Interaction logic for ViewNote.xaml
    /// </summary>
    public partial class ViewNote : Window
    {
        public ViewNote()
        {
            InitializeComponent();
            (DataContext as ListaNote).GetAllNote();
        }

        private void lblID_SelectedIndexChanged(object sender,
```

```
SelectionChangedEventArgs e)
{
    if (lblID.SelectedIndex != -1)
    {
        txtID.Text = (lblID.SelectedItem as Nota).Id.ToString();
        txtNota.Text = (lblID.SelectedItem as Nota).NotaStudent.ToString();
        txtDisciplina.Text = (lblID.SelectedItem as Nota).Disciplina;
        txtStudent.Text = (lblID.SelectedItem as Nota).Student;
        txtFlag.Text = (lblID.SelectedItem as Nota).FlagNota.ToString();
    }
}

public int GetDisciplinaCodDisciplina()
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetDisciplinaCodDisciplina", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramDenumire = new SqlParameter("@DENUMIRE", txtDisciplina.Text);

        cmd.Parameters.Add(paramDenumire);

        con.Open();

        reader = cmd.ExecuteReader();
        reader.Read();

        int codDisciplina;
        codDisciplina = (int)(reader[0]);
        return codDisciplina;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return -1;
    }
    finally
    {
        reader.Close();
    }
}

private int GetStudenNrMatricol()
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetStudentNrMatricol", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNume = new SqlParameter("@NAME", txtStudent.Text);

        cmd.Parameters.Add(paramNume);

        con.Open();
    }
}
```

```
        reader = cmd.ExecuteReader();
        reader.Read();

        int id;
        id = (int)(reader[0]);
        return id;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return -1;
    }
    finally
    {
        reader.Close();
    }
}

private int GetIdNota(int codDisciplina, int nrMatricol)
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetIdNota", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNota = new SqlParameter("@NOTA", int.Parse(txtNota.Text));
        var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);
        var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);
        var paramFlag = new SqlParameter("@FLAG", int.Parse(txtFlag.Text));

        cmd.Parameters.Add(paramNota);
        cmd.Parameters.Add(paramCodDisciplina);
        cmd.Parameters.Add(paramNrMatricol);
        cmd.Parameters.Add(paramFlag);

        con.Open();

        reader = cmd.ExecuteReader();
        reader.Read();

        int id;
        id = (int)(reader[0]);
        return id;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return -1;
    }
    finally
    {
        reader.Close();
    }
}

private void btnAdauga_Click(object sender, RoutedEventArgs e)
{
    int codDisciplina = GetDisciplinaCodDisciplina();
```

```
int nrMatricol = GetStudenNrMatricol();
SqlConnection con = DbConnector.Connection;
try
{
    SqlCommand cmd = new SqlCommand("AdaugaNota", con);
    cmd.CommandType = CommandType.StoredProcedure;

    var paramNota = new SqlParameter("@NOTA", txtNota.Text);
    var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);
    var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);
    var paramFlag = new SqlParameter("@FLAG_NOTA",
int.Parse(txtFlag.Text));

    cmd.Parameters.Add(paramNota);
    cmd.Parameters.Add(paramCodDisciplina);
    cmd.Parameters.Add(paramNrMatricol);
    cmd.Parameters.Add(paramFlag);

    con.Open();

    cmd.ExecuteNonQuery();
    MessageBox.Show("Nota has been added! Check DataBase!");
}
catch
{
    MessageBox.Show("Error In DataBase");
}
int id = GetIdNota(codDisciplina, nrMatricol);
(DataContext as ListaNote).Note.Add(new Nota()
{
    Id = id,
    NotaStudent = int.Parse(txtNota.Text),
    Disciplina = txtDisciplina.Text,
    Student = txtStudent.Text,
    FlagNota = int.Parse(txtFlag.Text)
}); ;
}

private void btnModifica_Click(object sender, RoutedEventArgs e)
{
    int codDisciplina = GetDisciplinaCodDisciplina();
    int nrMatricol = GetStudenNrMatricol();
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("ModificaNota", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNota = new SqlParameter("@NOTA", int.Parse(txtNota.Text));
        var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);
        var paramNrMatricol = new SqlParameter("@NR_MATRICOL", nrMatricol);
        var paramFlagNota = new SqlParameter("@FLAG_NOTA",
int.Parse(txtFlag.Text));
        var paramID = new SqlParameter("@ID", int.Parse(txtID.Text));

        cmd.Parameters.Add(paramNota);
        cmd.Parameters.Add(paramCodDisciplina);
        cmd.Parameters.Add(paramNrMatricol);
```

```
        cmd.Parameters.Add(paramFlagNota);
        cmd.Parameters.Add(paramID);

        con.Open();

        cmd.ExecuteNonQuery();

        MessageBox.Show("Nota has been updated! Check DataBase!");
    }
    catch
    {
        MessageBox.Show("Error in DataBase!");
    }

    var item = (DataContext as ListaNote).Note.FirstOrDefault(x => x.Id ==
int.Parse(txtID.Text));
    int index = (DataContext as ListaNote).Note.IndexOf(item);
    (DataContext as ListaNote).Note[index] = new Nota()
    {
        Id = int.Parse(txtID.Text),
        NotaStudent = int.Parse(txtNota.Text),
        Disciplina = txtDisciplina.Text,
        Student = txtStudent.Text,
        FlagNota = int.Parse(txtFlag.Text)
    };
}

private void btnSterge_Click(object sender, RoutedEventArgs e)
{
    int flag = 1;
    SqlConnection con = DbConnector.Connection;
    try
    {
        SqlCommand cmd = new SqlCommand("StergeNota", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramFlagNota = new SqlParameter("@FLAG_NOTA", flag);
        var paramID = new SqlParameter("@ID", int.Parse(txtID.Text));

        cmd.Parameters.Add(paramFlagNota);
        cmd.Parameters.Add(paramID);

        con.Open();

        cmd.ExecuteNonQuery();

        MessageBox.Show("Nota has been deleted(modified flag)! Check
DataBase!");
    }
    catch
    {
        MessageBox.Show("Error in DataBase!");
    }

    var item = (DataContext as ListaNote).Note.FirstOrDefault(x => x.Id ==
int.Parse(txtID.Text));
    int index = (DataContext as ListaNote).Note.IndexOf(item);
    (DataContext as ListaNote).Note[index] = new Nota()
    {
        Id = int.Parse(txtID.Text),
```

```
        NotaStudent = int.Parse(txtNota.Text),
        Disciplina = txtDisciplina.Text,
        Student = txtStudent.Text,
        FlagNota = flag
    };
}

private void btnBack_Click(object sender, RoutedEventArgs e)
{
    MainWindow mainWindow = new MainWindow();
    mainWindow.Show();
    this.Close();
}
}
```

ViewRaport.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace Catalog_Note
{
    /// <summary>
    /// Interaction logic for ViewRaport.xaml
    /// </summary>
    public partial class ViewRaport : Window
    {
        public ViewRaport()
        {
            InitializeComponent();
            (DataContext as ListaRaport).GetAllStudenti();
            //folosita pentru a nu putea schimba valoarea checkbox-ului
            checkBox.IsEnabled = false;
            //checkBox.IsChecked = true;
        }

        private void lblStudenti_SelectionChanged(object sender,
            SelectionChangedEventArgs e)
        {
            if (lblStudenti.SelectedIndex != -1)
            {
                List<string> aux = new List<string>();
                int nrMat = GetStudenNrMatricol((lblStudenti.SelectedItem as
                Student).Nume);
            }
        }
    }
}
```

```
//structura care permite doua valori de tpiri de date diferite
Tuple<int, double> nrCrediteSiMedie = GetNrCrediteSiMedie(nrMat);
int nrCredite = nrCrediteSiMedie.Item1;
double medie = nrCrediteSiMedie.Item2;

//pentru checkbox
if(GetIntegralist(nrMat)==true)
{
    checkBox.IsChecked = true;
}
else
{
    checkBox.IsChecked = false;
}

//pentru Disciplina si nota corespunzatoare
lblDisciplinaSiNota.ItemsSource = GetDisciplinaSiNotaPtStudent(nrMat);
//pentru numarul de credite
txtNrCredite.Text = nrCredite.ToString();
//pentru medie
txtMedia.Text = medie.ToString();
}
}

private int GetStudenNrMatricol(string student)
{
    SqlConnection con = DbConnector.Connection;
    SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetStudentNrMatricol", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramName = new SqlParameter("@NAME", student);

        cmd.Parameters.Add(paramName);

        con.Open();

        reader = cmd.ExecuteReader();
        reader.Read();

        int id;
        id = (int)(reader[0]);
        return id;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return -1;
    }
    finally
    {
        reader.Close();
    }
}

public string GetDisciplina(int codDisciplina)
{

```

```
SqlConnection con = DbConnector.Connection;
SqlDataReader reader = null;
try
{
    SqlCommand cmd = new SqlCommand("GetDisciplina", con);
    cmd.CommandType = CommandType.StoredProcedure;

    var paramCodDisciplina = new SqlParameter("@COD_DISCIPLINA",
codDisciplina);

    cmd.Parameters.Add(paramCodDisciplina);

    con.Open();

    reader = cmd.ExecuteReader();
    reader.Read();

    string disciplina;
    disciplina = reader.GetString(0);
    return disciplina;
}
catch
{
    MessageBox.Show("Error In DataBase");
    return "";
}
finally
{
    reader.Close();
}
}
public List<string> GetDisciplinaSiNotaPtStudent(int nr_matricol)
{
    SqlConnection con = DbConnector.Connection;
    List<string> disciplineSiNota = new List<string>();
    //SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetDisciplinaSiNotaPtStudent", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNume = new SqlParameter("@STUDENT", nr_matricol);

        cmd.Parameters.Add(paramNume);

        con.Open();

        //reader = cmd.ExecuteReader();
        //reader.Read();

        //int codDisciplina, nota;
        //codDisciplina = (int)(reader[0]);
        //nota = (int)(reader[1]);
        int codDisciplina;
        int nota;
        string disciplina;
        string disciplinaSiNota;
        SqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())
        {
```

```
        codDisciplina = (int)(reader[0]);
        nota = (int)(reader[1]);
        disciplina = GetDisciplina(codDisciplina);
        disciplinaSiNota = disciplina + ", " + nota.ToString();
        disciplineSiNote.Add(disciplinaSiNota);
    }
    reader.Close();
    return disciplineSiNote;
}
catch
{
    MessageBox.Show("Error In DataBase");
    return null;
}
}

public Tuple<int,double> GetNrCrediteSiMedie (int nr_matricol)
{
    SqlConnection con = DbConnector.Connection;
    var noteSicredite = new List<Tuple<int, int>>();

    //SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetCrediteSiNotePtStudent", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNume = new SqlParameter("@nr_matricol", nr_matricol);

        cmd.Parameters.Add(paramNume);

        con.Open();

        int nota;
        int nrCredite;
        SqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            nota = (int)(reader[0]);
            nrCredite = (int)(reader[1]);
            noteSicredite.Add(new Tuple<int, int>(nota, nrCredite));
        }
        reader.Close();
        //return noteSicredite;
        int totalCredite = 0;
        int sumCredite = 0;

        for(int i=0;i<noteSicredite.Count;i++)
        {
            totalCredite += noteSicredite[i].Item1 * noteSicredite[i].Item2;
            sumCredite += noteSicredite[i].Item2;
        }
        double medie = totalCredite / sumCredite;
        Tuple<int, double> nrCrediteSiMedie = new Tuple<int,
double>(totalCredite, medie);
        return nrCrediteSiMedie;
    }
    catch
    {

```

```
        MessageBox.Show("Error In DataBase");
        return null;
    }

}

//bool? - permite returnarea valorii null
public bool? GetIntegralist(int nrMatricol)
{
    List<int> listaDeNote = new List<int>();
    SqlConnection con = DbConnector.Connection;
    //SqlDataReader reader = null;
    try
    {
        SqlCommand cmd = new SqlCommand("GetNotePentruStudent", con);
        cmd.CommandType = CommandType.StoredProcedure;

        var paramNrMatricol = new SqlParameter("@nr_matricol", nrMatricol);

        cmd.Parameters.Add(paramNrMatricol);

        con.Open();

        SqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            listaDeNote.Add((int)(reader[0]));
        }
        reader.Close();

        for(int i=0;i<listaDeNote.Count;i++)
        {
            if(listaDeNote[i]<5)
            {
                return false;
            }
        }
        return true;
    }
    catch
    {
        MessageBox.Show("Error In DataBase");
        return null;
    }
}

private void btnBack_Click(object sender, RoutedEventArgs e)
{
    MainWindow mainWindow = new MainWindow();
    mainWindow.Show();
    this.Close();
}
}
```

1.3. Rezultate

Rezultatul va fi prezentat sub formă de videoclip, care se află în fișierul „Demo_Proiect.ZIP” conținând testarea fiecărei funcționalități a programului.